FSCT 8561 Security Applications

Midterm Exam Operation Sentinel: Secure Server Assessment

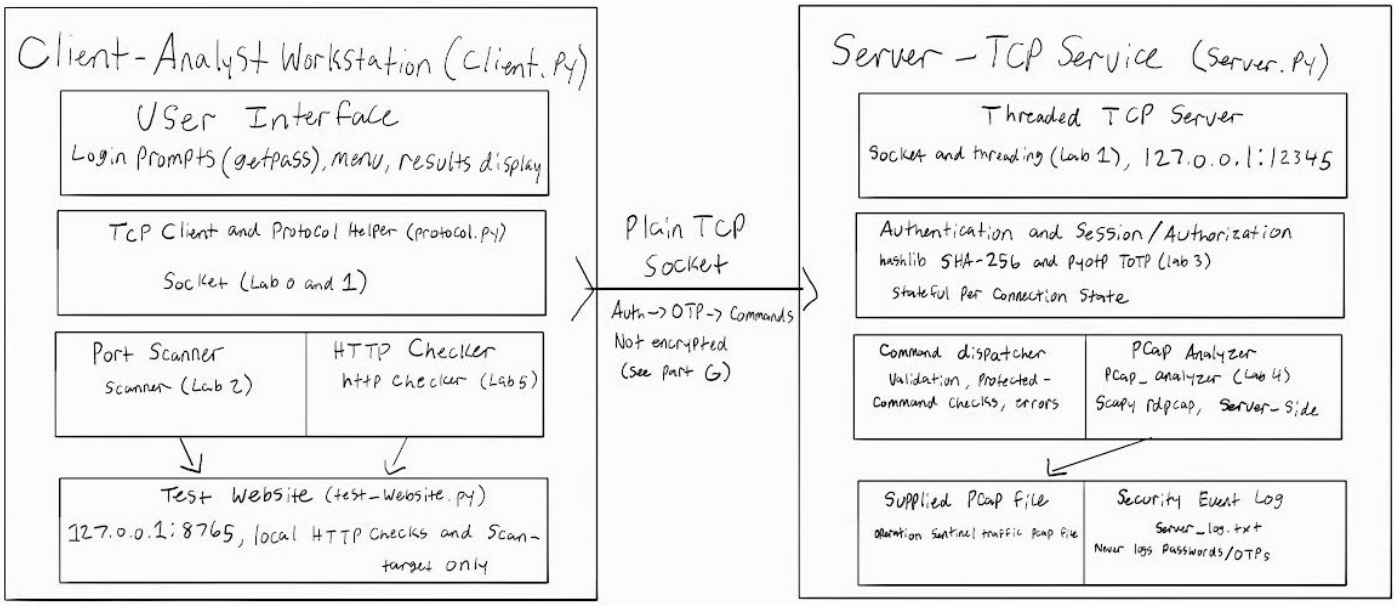
Ryan Lee A01365270

Instructor: Dr. Maryam R. Aliabadi

Part A: Architecture Design

Diagram

Operation Sentinel System Architecture



Information Flow for the Diagram

1. The User logs in at the client
   - The client sends AUTH, and then OTP over the TCP connection
   - Then the server verifies the hash then the TOTP
   - And then finally marks the connection as authenticated

2. Authenticated user picks a menu item
   - Scan and HTTP check run on the client against 127.0.0.1 only
   - PCAP and Status are sent to the server

3. Server checks authorization before every protected command
   - Then runs PCAP analyzer on its local file
   - And returns the results over the same connection

4. Server logs security events with time, user and source address
   - (login success, failures, unauthorized attempts, PCAP requests, errors)
   - Never with passwords or OTP codes

Component Responsibilities

Component	Responsibilities
User Interface	It prompts for the username, password, OTPs. Also displays the menu (Scan, HTTP Check, PCAP analysis, status, exit), and prints results into a readable form.
Protocol Helper	The only place that knows how messages are framed on the wire. Used by both the client and server, so both of them cannot drift apart from each other.
TCP Client	It opens one TCP connection to the server. Takes care of the Auth to OTP sequence, sends one command at a time, and waits for the server's response before sending the next command.
Port Scanner	This is only on the client-side. It checks TCP ports on 127.0.0.1 and reports if each port is open/closed with its service name
HTTP Checker	This is only on the client-side. It uses requests against the supplied local test website and then requests / , /health , /redirect, and a missing path, then reports status codes, the server header, the cookie that is set and which cookie flags are present/missing, and which security response headers are missing.
Threaded TCP Server	Listens on 127.0.0.1:12345 and it accepts connections, handles each connection in its own thread so one analyst disconnecting does not stop the service for the next test or analyst
Authentication/MFA Module	Verifies the password against a stored SHA256 hash and then verifies a TOTP code. The connection is marked authenticated when both of those steps succeed
Authorization/Dispatcher	It parses each command, validates its format and checks (before running it) whether the command is protected and if the connection is authenticated. If it is an unauthenticated request it will return with ACCESS_DENIED

	and have it logged
PCAP Analyzer	This is only on the server-side. It opens the supplied capture with Scapy, counts the TCP/UDP packets, summarises sources, destination and ports using only the IP/TCP/UDP headers and applies the detection rules. It returns text lines and the server sends those lines back to the client
Security Event Log	Contains records of security relevant events such as connection opened/closed, login success/failure, OTP failure, unauthorized command attempts, PCAP requests, and malformed input errors. Each of these logs have a date & time, event type, username if it is known, and client address. Passwords and OTP codes are never written here.

Stateful Interaction Model Rationale

The stateful interaction model is where the state lives on the server, per connection. The server remembers things for each connected client such as: whether the password step has passed, which username that was for, whether the OTP step has passed and how many failed attempts have occurred.

I chose this because:

The protocol is a sequence and not a set of independent requests.

- OTP is only meaningful after a successful AUTH for a specific user, because the server must know who's TOTP secret to verify against.

Authorization depends on history.

- A protected command such as PCAP is allowed/denied based on what happened earlier on the same connection. If we used a stateless model then the client would have to present a credential or token with every command. We did not cover any token mechanism so far, so adding one would add a new mechanism I would have to design myself.

The cost of statefulness is accepted and bounded

- Server state is small, it is discarded when the connection is closed, and the disconnected client simply just logs in again. The known weakness of this is that there is no session token, so authentication is bound to the open TCP connection itself.

Part B: Adversarial Threat Model

The assets in this system are: The stored password hash and TOTP secret, authenticated access to the protected server commands (PCAP results and diagnostic status), the integrity of the PCAP analysis results, the security event log, the availability of the server, and the analyst's own machine (which runs the scanner and HTTP checker).

Area/Asset	Threat & Attack Vector	Impact	Mitigation Implemented	Residual Risk
Sockets, the TCP channel and the credentials crossing it	Sniffing/Man in the middle attacks on plain TCP. Anyone who can observe local traffic can read every message, including the username and password sent in AUTH, because the connection is not encrypted. An attacker can also modify or inject messages.	Credential disclosure, session manipulation, and because the attacker can easily log in using the analyst's credentials, they would also have access to all the other permissions, making every other security control useless	Scope reduction. The server binds to 127.0.0.1, and not a network interface, so the channel is not exposed to a LAN in this test setup. Passwords are never logged and are only stored as a hash on the server.	High. A real deployment would require transport encryption.
Authentication/ MFA, password and TOTP	Replay and brute force. An observer who captures an AUTH message can re-send it. The observer who can reach the port can also guess passwords or 6 digit codes repeatedly. The TOTP code would only have 1 million possibilities which is easily brute forced by a modern computer.	Account takeover of the analyst's account which would give full access to protected commands.	Sequence enforcement. An OTP is only accepted immediately after a successful password step for the same connection and user. A failed attempt limit will close the connection after 3 failed password/OTP attempts. Passwords are verified against a SHA256 hash and never stored in plaintext on the server	Medium. TOTP codes remain replayable within their 30 second window by an attacker who can observe the plain TCP channel, and there is no per account lockout across new connections. An attacker can reconnect again and again.
Sockets / authorization, protected server functions	Unauthenticated or unauthorized command use. A client that skips login and sends PCAP or STATUS directly. A client that sends commands out of order tries to obtain diagnostic information or PCAP results without completing the MFA	Disclosure of internal diagnostic info and traffic analysis results to an unauthenticated party. This would defeat the purpose of the administrative interface.	Server-side authorization check before every protected command based on the per connection authenticated state. The client is never trusted to enforce this. Rejection is showed as ACCESS_DENIED and the attempt is logged with source address	Low to Medium. The check itself is simple and tested, but it protects a connection and not a person. Anyone that is using an already authenticated connection inherits its access. No idle timeout is implemented either

Scanning (Port scanner)	Misuse of the scanner/scanning unauthorized targets. The same code that checks the test website could be pointed at systems the user has no right to scan.	Legal/policy exposure for the analyst. Possible disruption or false findings and unreliable assessment results	Hard restriction in code where the scanner accepts localhost only and refuses any other target.	Low. The scanner intentionally cannot be reused on a real network without changing the code
Scapy-based analysis	Untrusted input file/parser abuse and over-claiming. The .pcap is input the server did not create. A malformed or hostile capture could crash the analyzer or be made to trigger misleading detections.	Loss of the PCAP function, false accusations or a false sense of security in the report, privacy exposure if packet payloads were collected unnecessarily	The analyzer reads the file offline with rdpcap(). File errors are caught and returned as a normal error response instead of crashing the server. The analyzer reads headers and counts only, and never payload content. Every detection is reported with its rule, threshold and the counts that triggered it, so the reader can judge the claim.	Medium. Analyzing hostile files using Scapy cannot be completely secured, and pattern detection is not foolproof. A flagged file should be a reason to double check, not absolute confirmation or proof of an attack.
HTTP Assessment (the test website and the checker)	False conclusions and information disclosure. The checker could report a missing header as a vulnerability, or an attacker could use the information the checker legitimately collects to fingerprint the service. A checker bug could also send requests somewhere other than the local test site.	A report that overstates/understates risk. Fingerprinting info is made easier to collect. Assessment traffic sent to an unintended target.	The target url is fixed in code to http://127.0.0.1:8765. Uses wording discipline “Configuration Review” and “Informational” and each finding states it does not by itself prove an exploitation	Low to medium. The wording discipline reduces over claiming but cannot prevent a reader from over interpreting a finding.
Sockets/server availability and log integrity	Malformed input, flooding and log abuse. A client can send messages with no delimiter, missing or extra fields, oversized content, garbage bytes, or simply disconnect mid-message. Rapid repeated connections can waste server threads. Text taken from the network and written into the log could also be used to inject fake log lines.	Denial of service of the assessment tool itself. A corrupted or misleading security log.	An unterminated message is rejected with ERROR, and not processed. Every command is field-count and content is validated before use. Unknown commands get ERROR. Decoding errors and connection resets are caught per connection so one bad client cannot kill the threaded server.	Medium. There is no connection rate limiting and no cap on simultaneous connections, so a determined local flooding attack can still degrade the server.

			Log entries are written by the server in a fixed format from validated fields only. Usernames are restricted to a short alphanumeric format at AUTH, which blocks newline log injection through the username. Every rejection and disconnect is logged.	

Part C: Protocol Design

Transport and framing

TCP, server 127.0.0.1:12345, one connection per analyst session. Framing is one message per line, terminated by \n. TCP is a stream, so one recv() does not equal one message, the receiver accumulates bytes until the newline before processing which handles messages split across segments and two messages arriving together. The max message length is 4096 bytes. Longer or unterminated input is rejected and the partial message is discarded, field format is COMMAND|field|field. The one limitation is that fields cannot contain | or a newline, so passwords containing | are not supported by this teaching protocol.

Message Catalogue

Message	Fields	State Required	Success	Failure
AUTH| username| password	Exactly 3	none	OTP_REQUIRED	ACCESS_DENIED
OTP|code	Exactly 2, 6 digits	password step passed, this connection	ACCESS_GRANTED	ACCESS_DENIED
PING	Exactly 1	authenticated	OK|PONG	ACCESS_DENIED
STATUS	Exactly 1	authenticated	OK|User: <name> | PCAP file: <name> | Server: Operation Sentinel	ACCESS_DENIED
PCAP	Exactly 1	authenticated	PCAP_BEGIN, one	ACCESS_DENIED

			PCAP| <line> per result line, PCAP_END	or ERROR|PCAP analysis failed
EXIT	Exactly 1	any	OK|Goodbye, then the server closes the connection	
Anything else				ERROR|Unknown command

The PCAP_BEGIN … PCAP_END exists so the client knows the complete multi-line result has arrived before prompting again. The results are never sent as one message with embedded newlines.

Authentication and MFA sequence

Client Server
| -------- AUTH|alice|<password> ---------> | SHA-256(password) == stored hash?
| <------------- OTP_REQUIRED ------------- | password step marked passed (alice)
| ------------ OTP|<6-digit> --------------------> | pyotp verify(code, valid_window=1)
| <----------- ACCESS_GRANTED ---------- | connection marked authenticated
| -------- PCAP / STATUS / PING ----------> | each re-checked against that state

Rules:

- An OTP before a successful AUTH is always ACCESS_DENIED
- A second AUTH on an authenticated connection gets ERROR|Already authenticated
- 3 failed AUTH or OTP attempts make the server send ACCESS_DENIED, log it and close that connection
- The server console prints the current TOTP code for the test account because analyst and operator are the same person in this setup

Authorization Rules

Protected: PING, STATUS, PCAP. Public: AUTH, OTP, EXIT. The check runs on the server, on every protected command, every time. Never once at login and never on the client, because a hand written client can send anything in any order. One role (analyst), one account, so authorization is binary. Authenticated connections get all protected commands, unauthenticated get none and no per-command permissions are claimed. A rejected command does not close the connection, but is logged with the source address.

Malformed Requests and Error Responses

Malformation	Server Behaviour
Wrong field count for the command	ERROR|Malformed <COMMAND> message
Empty or non-alphanumeric / over-long username	ERROR|Invalid username format
Empty password field	Treated as a failed login: ACCESS_DENIED (not an error and avoids leaking validation detail)
OTP not exactly 6 digits	ERROR|OTP must be 6 digits
Unknown command verb	ERROR|Unknown command
Over 4096 bytes, or connection ends mid-message	ERROR|Message too long, the partial message is discarded and that connection is closed
Non-UTF-8 bytes	Decode error caught, logged, only that connection closed
Disconnect without EXIT	Caught, logged as unexpected disconnection, that thread ends and the server keeps running

Session Termination

Normal: EXIT -> OK|Goodbye, the server logs it, closes that connection, discards its state and keeps listening

Abnormal: Resets, client crashes, or over-limit authentication failure. The connection closes and the observed reason is logged.

Replay and Security Limitations

The protocol provides no confidentiality and no integrity protection. The password sent in AUTH crosses the socket in plaintext, and so does every command and every result. TOTP authenticates the user at login. It does not protect the channel. Replay is also possible. A captured AUTH message can be sent again exactly as it was captured, and it will pass the password step. This is because the password is a static secret, and the protocol has no nonce or challenge that changes from one session to the next. A captured OTP code also works for the rest of its validity window of about 30 seconds. The valid_window=1 setting widens that window a little, because the server also accepts the neighbouring time step to allow for clock

differences. Inside a connection there are no sequence numbers and no message authentication codes. A replayed command therefore looks the same as a fresh one. Authentication is tied to the connection itself rather than to a token. Whoever holds the connection holds the session.

Implemented vs Proposed

Implemented in this submission: password verification with SHA-256 and MFA with TOTP. An authorization check that runs on the server for every protected command. Framing with the 4096 byte cap, field validation and uniform error replies. A limit on failed attempts for each connection. Event logging that never includes secrets. A server that binds to localhost only, and a scanner that accepts localhost targets only.

Proposed for a production version: TLS for the transport. Salted slow password hashing with Argon2, bcrypt, scrypt or PBKDF2 which is the storage of the Authentication lecture names for production, together with an account lockout that applies across connections. A nonce or a session token, so that a captured login cannot be replayed. Rate limiting, an idle timeout and protected storage for the log. TOTP enrolment through an authenticator app instead of the code printed by the test server.

Part D: Component Design

Component	Inputs	Outputs	Expected Behaviour	Failure Conditions	Security Controls
Port Scanner	Target (127.0.0.1 or localhost only) and start and end ports, checked before scanning	One line per open port with its service name, and the list of open ports, or a line saying none were found	TCP connect scan using connect_ex() with a 0.5 second timeout. Return code 0 means open. Ports are checked in order	Ports that are not numbers, outside 1 to 65535, start above end, or a span over 1000. A message is shown and no scan runs	Target locked to localhost in code. Span capped at 1000 ports. Normal connection attempts only. Results reported as open or closed only
MFA Module	Username and password from AUTH, then a six digit code from OTP	OTP_REQUIRED, ACCESS_GRANTED or ACCESS_DENIED. One log line per attempt, without the password or the code	Password is hashed with SHA 256 and compared with the stored hash. The code is checked with pyotp, valid_window=1. Both steps must pass, in order, on the same connection	Unknown user and wrong password both give ACCESS_DENIED. An OTP before the password step is denied. Three failed attempts of either kind close the	Password kept as a hash only and never logged. Replies do not reveal whether a username exists. No message can skip the OTP step

				connection	
HTTP Checker	None from the user. The target is fixed in code to the local test website on port 8765. It requests the root path, health, redirect and a missing path with requests and a timeout of 5 seconds	Status code for each path. For the root path, the Server header, the cookie flags and the missing header results. Findings printed as Category, Endpoint, Observation and Review Status	Uses the requests library with a 5 second timeout. Reports what each path returns, including the redirect target	Website not running gives a could not connect finding and the checker still finishes. An unexpected status code is reported as seen	Target fixed to localhost. GET requests only. Missing headers reported as configuration findings, not as proven vulnerabilities. HSTS is shown as a note only, not a finding
PCAP Analyzer	The path of the supplied capture file on the server. The client cannot choose the file	Packet total and duration, TCP and UDP counts, counts by source, destination and port, then any alerts with the numbers behind them	Offline read with rdpcap() in time order. Alert rules: more than 10 SYN packets to different ports from one source, more than 10 SYN packets to one port with no completed handshake, more than 20 packets from one source in 5 seconds	Missing or unreadable file gives an error reply and the server stays up. If no rule matches, the summary is returned with a no suspicious patterns line	Offline file only. No live capture and no packets sent. Payload content is never read. Alert lines include the counts that triggered them

Sensitive information intentionally not collected or logged

Packet payload content and HTTP response bodies beyond their length and headers are not collected. This is because the headers answer every question the tool asks. Payloads can hold credentials and personal data, so collecting them adds on risk with no benefit of having them. Leaving them out keeps the output safe to store and submit.

Part E: Full Integrated Implementation

File	Role	Lab Source
protocol.py	Newline framing and the message cap, with the send and receive functions both sides use	Lab 0 and 1
server.py	Threaded server on port 12345, password and TOTP login, session state, the authorization check, command handling and logging	Lab 1 and 3
client.py	Login prompts, the menu, and the results display. Runs the scanner and the HTTP checker locally	Labs 0-3
scanner.py	TCP connect scan with the Lab 2 checks, and the localhost only rule	Lab 2
http_checker.py	The four path checks on the test website, with status codes, the server header, cookie flags and missing headers	Lab 5
pcap_analyzer.py	Offline scapy read of the capture, the summary counts and the three alert rules	Lab 4
test_website.py	Required test component given to me	
requirements.txt	Provides a list of all libraries used	
README.md	Instructions for the code	

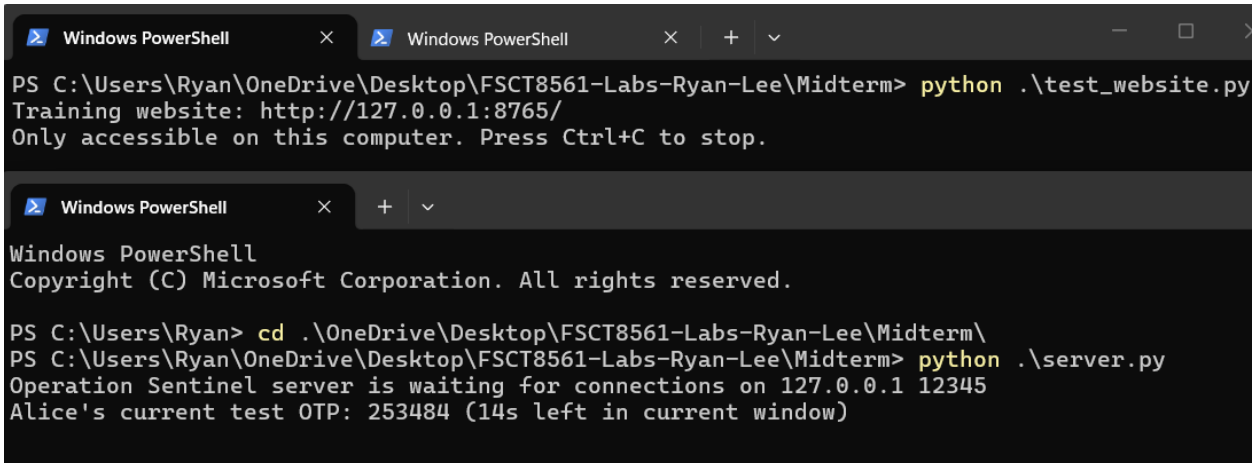
Why the key controls exist: The authorization gate exists because the client is attacker-controlled. A hand written client can send any command in any order so only a server side check means anything. Framing with a size cap exists because TCP does not deliver messages in recv() sized pieces and unbounded buffering is a denial of service hole. Attempt

limits exist because a 6 digit code is easily guessable/brute forced. The log excludes secrets because a log containing passwords or codes would be a big vulnerability issue.

Part F: Comprehensive Testing

Setup

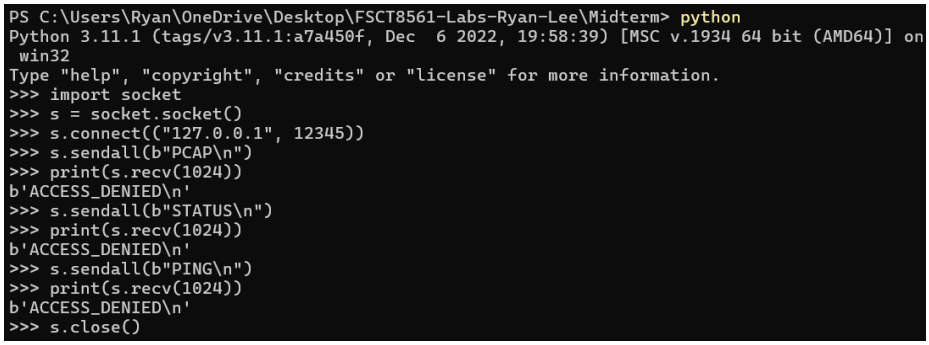
The test environment running before testing. The top window shows the provided test website serving on 127.0.0.1 port 8765, and the bottom window shows the Operation Sentinel server waiting for connections on 127.0.0.1 port 12345.



Tests

The first 4 tests we will do are tests that are manually typed through the python shell.

Test 1: Unauthorized Commands
Expected ACCESS_DENIED for all three protected commands sent with no login. The screenshot shows PCAP, STATUS and PING each answered with ACCESS_DENIED at a hand typed Python prompt. The result is a pass.



Test 2: OTP sent before AUTH

Expected ACCESS_DENIED when a code is sent before any password step. The screenshot shows the OTP message answered with ACCESS_DENIED. The result is a pass.

```
>>> s = socket.socket()
>>> s.connect(("127.0.0.1", 12345))
>>> s.sendall(b"OTP|123456\n")
>>> print(s.recv(1024))
b'ACCESS_DENIED\n'
>>> s.close()
>>>
```

Test 3: Invalid protocol inputs

Expected an ERROR reply for each malformed message, with the server still running. The screenshot shows ERROR replies for the unknown commands and for the AUTH message with a missing field. The result is a pass.

```
>>> s = socket.socket()
>>> s.connect(("127.0.0.1", 12345))
>>> s.sendall(b"HELLO\n")
>>> print(s.recv(1024))
b'ERROR|Unknown command\n'
>>> s.sendall(b"FOO|bar\n")
>>> print(s.recv(1024))
b'ERROR|Unknown command\n'
>>> s.sendall(b"AUTH|alice\n")
>>> print(s.recv(1024))
b'ERROR|Malformed AUTH message\n'
>>> s.close()
>>>
```

Test 4: Over long message

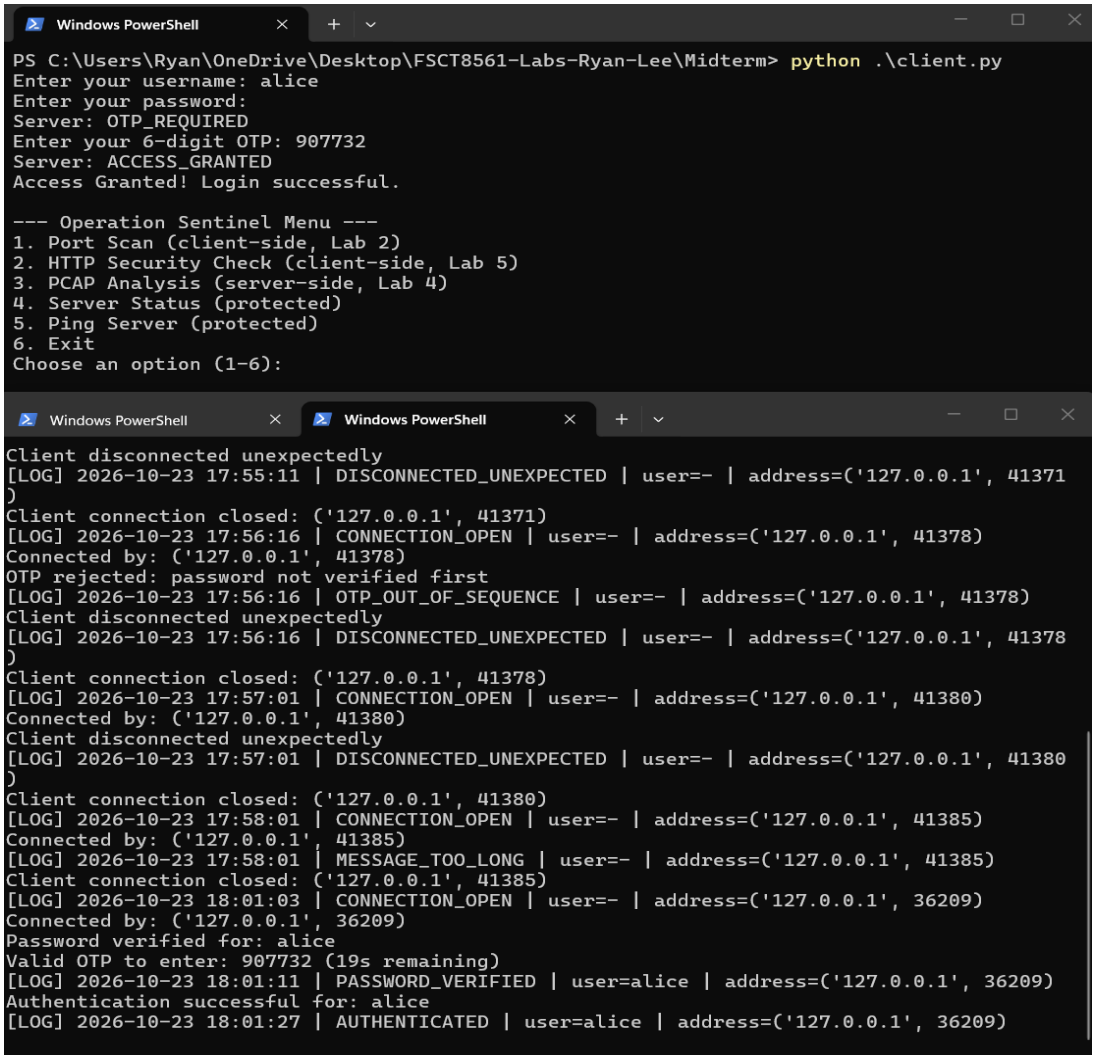
Expected the server to reject a message over the 4096 byte limit and close the connection. The screenshot shows ERROR, Message too long, and the second read returning empty, which confirms the server closed the connection. The result is a pass.

```
>>> s = socket.socket()
>>> s.connect(("127.0.0.1", 12345))
>>> s.sendall(b"A" * 5000 + b"\n")
>>> print(s.recv(1024))
b'ERROR|Message too long\n'
>>> print(s.recv(1024))
b''
>>> s.close()
>>> exit()
```

Test 5: Successful Login

Expected OTP_REQUIRED after the correct password, then ACCESS_GRANTED after the correct code. The screenshot shows both replies in the client, followed by the menu. The result is a pass.

The bottom terminal shows the server console during the same login. The log lines show the password verified and the user authenticated events for alice.



Test 6: Commands after Login:
Expected the protected commands to work once authenticated. The screenshot shows PING answered with OK, PONG and STATUS answered with a reply naming the logged in user alice. The result is a pass

```
--- Operation Sentinel Menu ---
1. Port Scan (client-side, Lab 2)
2. HTTP Security Check (client-side, Lab 5)
3. PCAP Analysis (server-side, Lab 4)
4. Server Status (protected)
5. Ping Server (protected)
6. Exit
Choose an option (1-6): 5
Server: OK|PONG

--- Operation Sentinel Menu ---
1. Port Scan (client-side, Lab 2)
2. HTTP Security Check (client-side, Lab 5)
3. PCAP Analysis (server-side, Lab 4)
4. Server Status (protected)
5. Ping Server (protected)
6. Exit
Choose an option (1-6): 4
Server: OK|User: alice | PCAP file: FSCT8561_Operation_Sentinel_Traffic.pcap | Server: Ope
ration Sentinel
```

Test 7 and 8: Port Scans

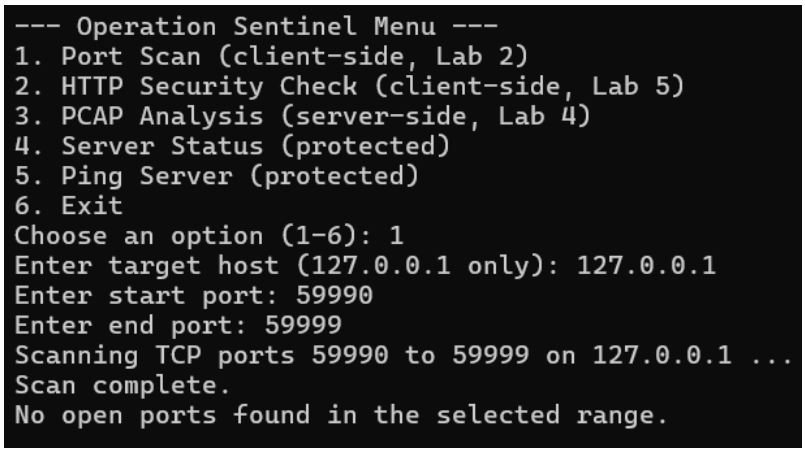
Expected the scan of ports 8700 to 8800 to find the test website. The screenshot shows port 8765 reported open and one open port found. The result is a pass.

```
--- Operation Sentinel Menu ---
1. Port Scan (client-side, Lab 2)
2. HTTP Security Check (client-side, Lab 5)
3. PCAP Analysis (server-side, Lab 4)
4. Server Status (protected)
5. Ping Server (protected)
6. Exit
Choose an option (1-6): 1
Enter target host (127.0.0.1 only): 127.0.0.1
Enter start port: 8700
Enter end port: 8800
Scanning TCP ports 8700 to 8800 on 127.0.0.1 ...
Port 8765 is OPEN - unknown
Scan complete.
1 open port(s) found.
Open ports: [8765]
```

Expected the scan of ports 12340 to 12350 to find the server. The screenshot shows port 12345 reported open and one open port found. The result is a pass.

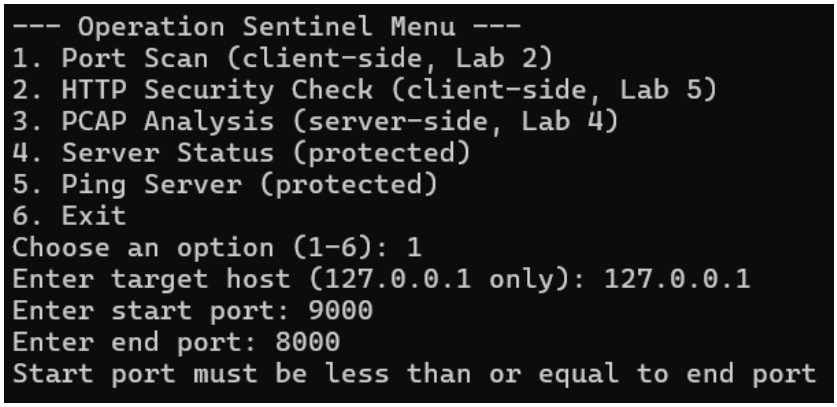
```
--- Operation Sentinel Menu ---
1. Port Scan (client-side, Lab 2)
2. HTTP Security Check (client-side, Lab 5)
3. PCAP Analysis (server-side, Lab 4)
4. Server Status (protected)
5. Ping Server (protected)
6. Exit
Choose an option (1-6): 1
Enter target host (127.0.0.1 only): 127.0.0.1
Enter start port: 12340
Enter end port: 12350
Scanning TCP ports 12340 to 12350 on 127.0.0.1 ...
Port 12345 is OPEN - unknown
Scan complete.
1 open port(s) found.
Open ports: [12345]
```

Expected the scan of ports 59990 to 59999 to find nothing, because no service listens there. The screenshot shows no open ports found in the selected range. The result is a pass.



Test 9: Invalid Scanner Input

Expected the client to refuse a range where the start port is higher than the end port. The screenshot shows the message that the start port must be less than or equal to the end port, and no scan was run. The result is a pass.



Test 10: HTTP security findings

Expected the checker to report the weaknesses of the test website. The screenshot shows the plain HTTP finding, the Server header value, the cookie missing the Secure, HttpOnly and SameSite flags, and the two missing security headers, with HSTS shown as a note only. The result is a pass.

```
--- Operation Sentinel Menu ---
1. Port Scan (client-side, Lab 2)
2. HTTP Security Check (client-side, Lab 5)
3. PCAP Analysis (server-side, Lab 4)
4. Server Status (protected)
5. Ping Server (protected)
6. Exit
Choose an option (1-6): 2
HTTP security checks against http://127.0.0.1:8765
GET / returned status 200 (90 bytes)
Category:      Configuration Review
Observation:   The service is offered over plain HTTP, not HTTPS
Review Status: Transport is unencrypted; review configuration. This alone does not prove exploitation
----------------------------------------
Category:      Informational
Endpoint:      /
Observation:   Server header exists: FSCT8561LabHTTP/1.0
Review Status: Informational; implementation detail, not automatically high severity
----------------------------------------
Category:      Configuration Review
Endpoint:      /
Observation:   Set-Cookie is missing flag(s): Secure, HttpOnly, SameSite
Review Status: Review configuration; this alone does not prove exploitation
----------------------------------------
Category:      Configuration Review
Endpoint:      /
Observation:   Header 'Content-Security-Policy' is missing or empty
Review Status: Review configuration; this alone does not prove exploitation
----------------------------------------
Category:      Configuration Review
Endpoint:      /
Observation:   Header 'X-Content-Type-Options' is missing or empty
Review Status: Review configuration; this alone does not prove exploitation
----------------------------------------
Category:      Informational
Endpoint:      /
Observation:   Strict-Transport-Security is not present; not counted as a finding because this is an HTTP-only local test service and HSTS applies to HTTPS deployments
Review Status: Informational; review only if this service is ever deployed over HTTPS
----------------------------------------
GET /health (health endpoint) returned status 200
GET /redirect (redirect behaviour) returned status 302 with Location: /
GET /missing (missing page handling) returned status 404
HTTP checks complete. Findings are configuration observations, not proof of exploitability.
```

Test 11: Wrong Password

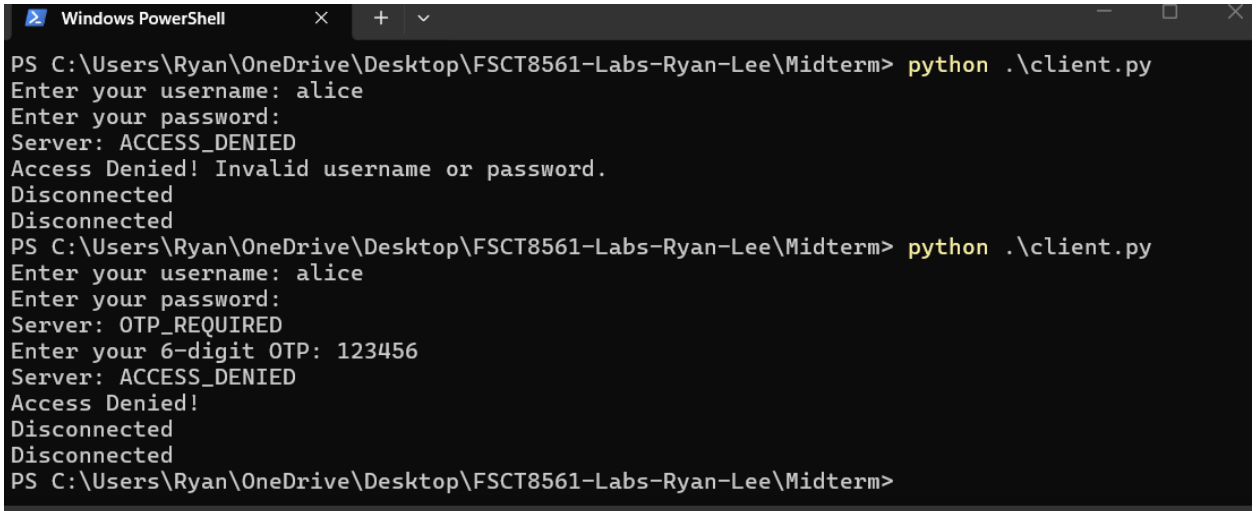
Expected the login to be refused. The screenshot shows the client receiving ACCESS_DENIED and disconnecting. The result is a pass.

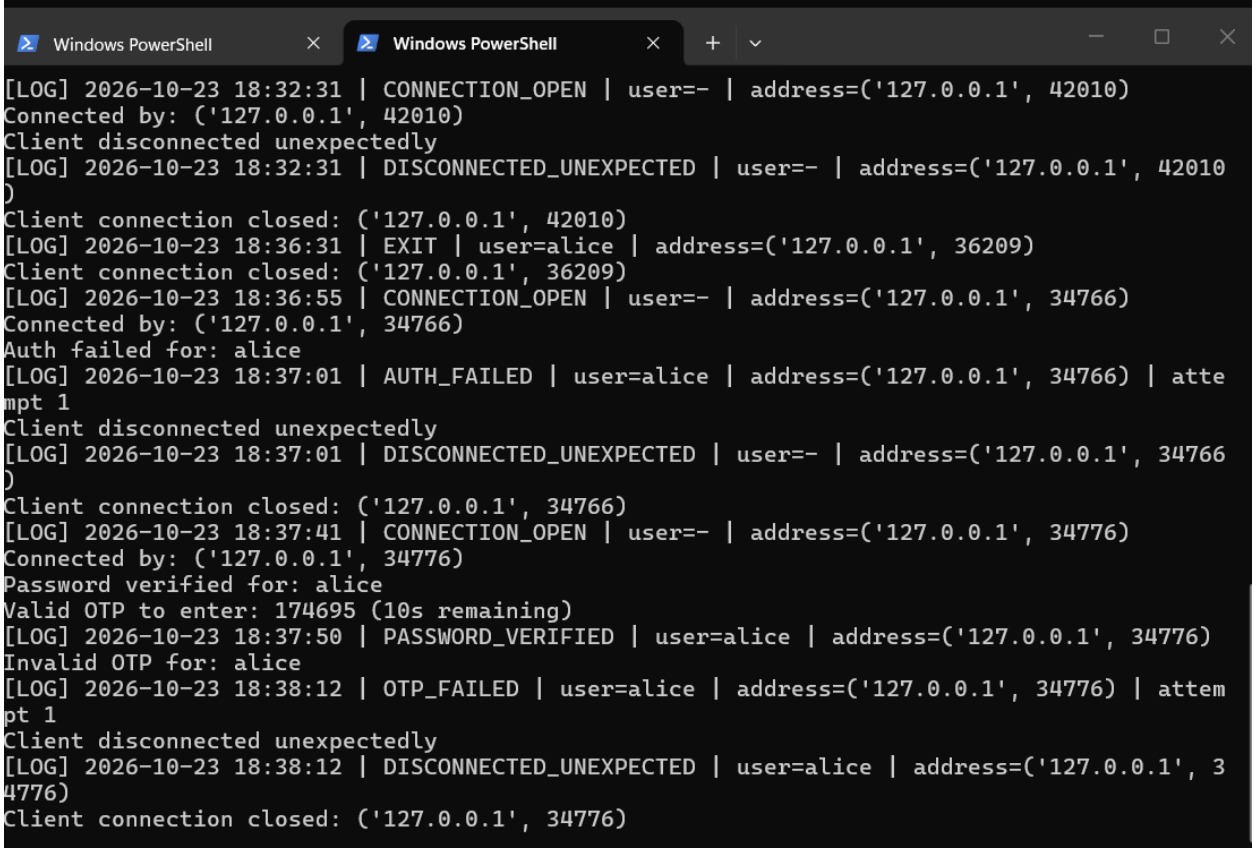
```
PS C:\Users\Ryan\OneDrive\Desktop\FSCT8561-Labs-Ryan-Lee\Midterm> python .\client.py
Enter your username: alice
Enter your password:
Server: ACCESS_DENIED
Access Denied! Invalid username or password.
Disconnected
Disconnected
PS C:\Users\Ryan\OneDrive\Desktop\FSCT8561-Labs-Ryan-Lee\Midterm>
```

Test 12: Wrong OTP

Expected the login to be refused when the wrong code is entered after the correct password. The screenshot shows OTP_REQUIRED first, then ACCESS_DENIED after the wrong code was submitted. The result is a pass.

The bottom terminal shows the server console for the wrong code attempt. The log shows the OTP_FAILED event for alice, and no code value is recorded in the log





Test 13: HTTP connection failure
Expected the checker to report a connection failure with the test website stopped. The screenshot shows the checker reporting that it could not connect to 127.0.0.1 port 8765 and advising that the test website is not running. The result is a pass.

The bottom terminal shows the test website terminal after it was stopped for this test, showing the requests it served earlier in the session.

```
--- Operation Sentinel Menu ---
1. Port Scan (client-side, Lab 2)
2. HTTP Security Check (client-side, Lab 5)
3. PCAP Analysis (server-side, Lab 4)
4. Server Status (protected)
5. Ping Server (protected)
6. Exit
Choose an option (1-6): 2
HTTP security checks against http://127.0.0.1:8765
Could not connect to http://127.0.0.1:8765: HTTPConnectionPool(host='127.0.0.1', port=8765
): Max retries exceeded with url: / (Caused by NewConnectionError('<urllib3.connection.HTT
PConnection object at 0x0000019CB386A550>: Failed to establish a new connection: [WinError
 10061] No connection could be made because the target machine actively refused it'))
Check that the test website is running (python test_website.py) and try again.
```

Windows PowerShell | Windows PowerShell

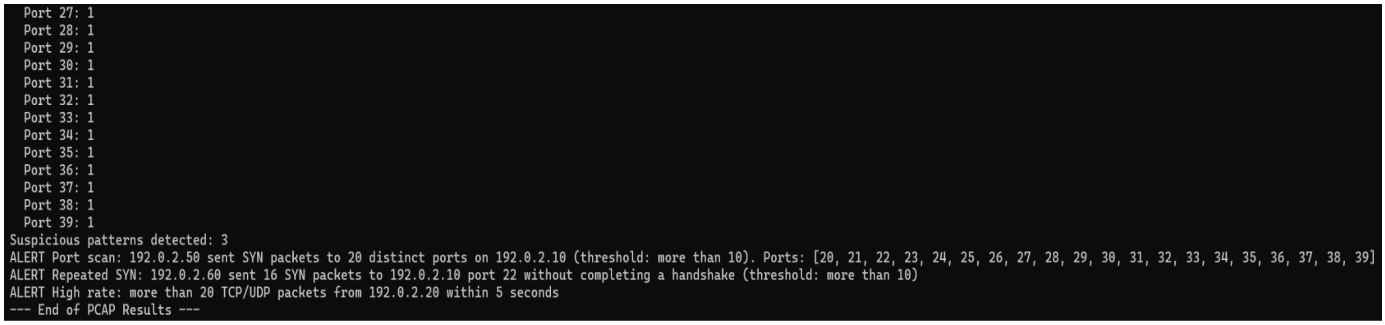
```
PS C:\Users\Ryan\OneDrive\Desktop\FSCT8561-Labs-Ryan-Lee\Midterm> python .\test_website.py
Training website: http://127.0.0.1:8765/
Only accessible on this computer. Press Ctrl+C to stop.
[HTTP test] 127.0.0.1 - "GET / HTTP/1.1" 200 -
[HTTP test] 127.0.0.1 - "GET /health HTTP/1.1" 200 -
[HTTP test] 127.0.0.1 - "GET /redirect HTTP/1.1" 302 -
[HTTP test] 127.0.0.1 - "GET /missing HTTP/1.1" 404 -
PS C:\Users\Ryan\OneDrive\Desktop\FSCT8561-Labs-Ryan-Lee\Midterm>
```

Test 14: PCAP summary and detection

Expected the analyzer to summarize the capture. The screenshot shows the totals of 123 packets, 113 TCP and 10 UDP, with the counts by source, destination and destination port. The result is a pass.

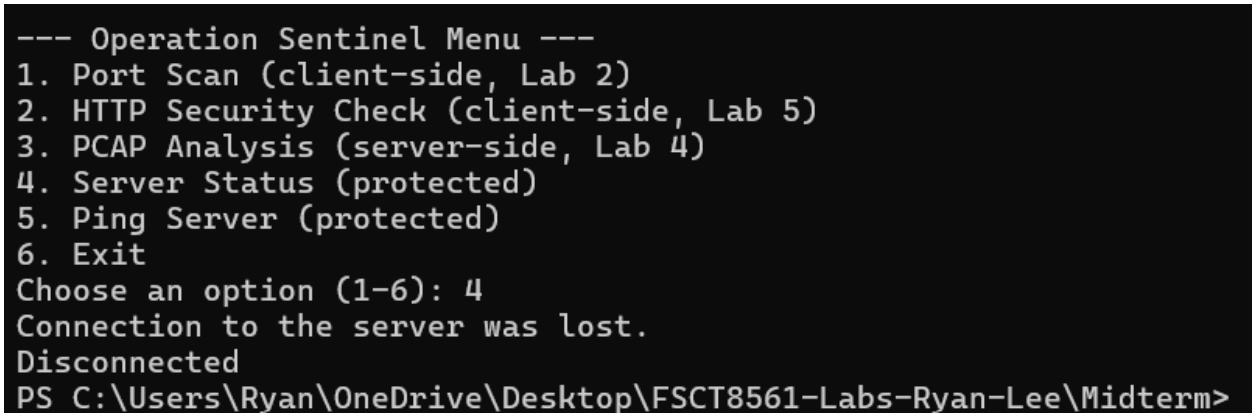
```
--- Operation Sentinel Menu ---
1. Port Scan (client-side, Lab 2)
2. HTTP Security Check (client-side, Lab 5)
3. PCAP Analysis (server-side, Lab 4)
4. Server Status (protected)
5. Ping Server (protected)
6. Exit
Choose an option (1-6): 3
--- PCAP Analysis Results (from server) ---
Total packets: 123 over 17.82 seconds
Total TCP packets: 113
Total UDP packets: 10
Packets by source:
  192.0.2.20: 63
  192.0.2.10: 24
  192.0.2.50: 20
  192.0.2.60: 16
Packets by destination:
  192.0.2.10: 99
  192.0.2.20: 24
Packets by destination port:
  Port 5000: 48
  Port 22: 17
  Port 53: 10
  Port 80: 5
  Port 40000: 1
  Port 40001: 1
  Port 40002: 1
  Port 40003: 1
  Port 40004: 1
  Port 40005: 1
  Port 40006: 1
  Port 40007: 1
  Port 40008: 1
  Port 40009: 1
  Port 40010: 1
  Port 40011: 1
  Port 40012: 1
  Port 40013: 1
  Port 40014: 1
  Port 40015: 1
  Port 40016: 1
  Port 40017: 1
  Port 40018: 1
  Port 40019: 1
  Port 40020: 1
  Port 40021: 1
  Port 40022: 1
  Port 40023: 1
  Port 20: 1
  Port 21: 1
  Port 23: 1
  Port 24: 1
  Port 25: 1
  Port 26: 1
```

Expected the analyzer to flag the suspicious sources in the capture. The screenshot shows three patterns detected, with alerts for the port scan from 192.0.2.50, the repeated SYN packets from 192.0.2.60, and the high packet rate from 192.0.2.20. The result is a pass.



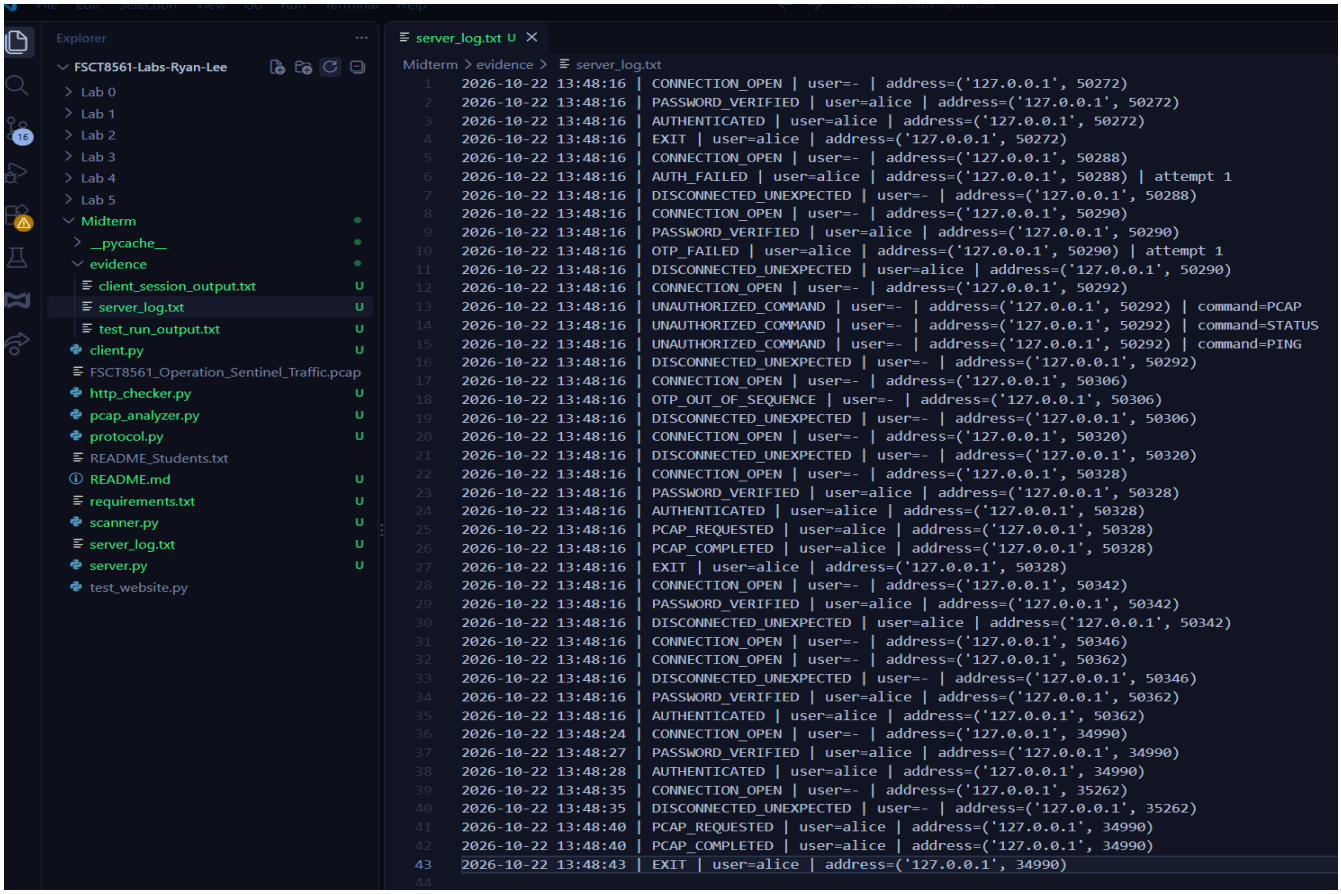
Test 15: Server disconnection

Expected the client to report the lost connection when the server is closed mid session. The screenshot shows the client printing that the connection to the server was lost, then disconnecting cleanly. The result is a pass.



Test 16: Log integrity

Expected the log to contain the events of the testing session and no secrets. The screenshot shows the server log file with the recorded events from the tests above. Searching the file for the password and for a code used during testing returned no matches. The result is a pass.



Part G: Security Analysis and Reflection

9. Which threats from your Day 1 model did your implementation mitigate, and which remain?

With the implementation, I mitigated:

- Unauthorized use of protected commands. The server checks the authenticated state on every request, and Test 1 in Part F shows PCAP, STATUS and PING all denied with no login, and denied again when the steps are done out of order in Test 2.
- Scanner misuse outside the authorized scope. The scanner refuses any target that is not localhost and refuses invalid ranges, shown in Test 9.
- Malformed input and flooding at the message level. The framing cap and error handling reject bad input without stopping the server, shown in Tests 3 and 4.
- Secrets leaking through the log. The log records events only, and Test 16 confirms it holds neither the password nor any code.

Partially mitigated:

- Brute force and replay against authentication. The connection closes after three failed passwords or three failed code attempts, and each code lives for about 30 seconds. What remains is that nothing stops an attacker from reconnecting and trying again, and there is no account lockout or delay between connections.
- Drawing false conclusions from the analyzers. The PCAP rules use thresholds declared in advance and the HTTP findings are worded as configuration review, which reduces over claiming. The parser still runs on the server, and a single capture can still be misread, as the low confidence alert in Question 10 shows.

Not mitigated:

- Eavesdropping and tampering on the channel itself. The connection is plain TCP with no encryption, so anyone who can observe the traffic can read it and can sit between the client and the server. This was the highest risk in Part B and it is unchanged.

10. What suspicious traffic did your PCAP analyzer identify? Cite packet-level or summary evidence.

The capture holds 123 packets over about 18 seconds, 113 TCP and 10 UDP, all addressed within 192.0.2.0 addresses, with 192.0.2.10 receiving 99 of the packets. The analyzer raised three alerts.

- 192.0.2.50, a port scan. It sent 20 TCP SYN packets to 20 different ports on 192.0.2.10, ports 20 to 39 in order, inside 0.86 seconds, and received no replies. One SYN each to many sequential ports in under a second is scanning behaviour and not normal use.
- 192.0.2.60, repeated connection attempts to one service. It sent 16 SYN packets to port 22 in about half a second and never completed a single handshake. No connection was ever established. That pattern fits repeated probing of the SSH port.
- 192.0.2.20, high packet rate and low confidence. It tripped the rate rule with 21 packets inside one 5 second window. On inspection its traffic is mostly 24 completed connections to port 5000 with full handshakes, plus DNS queries. Completed handshakes look like a busy legitimate client, so this alert is reported with low confidence and shows why a rate rule on its own is not proof of an attack.

11. What did your HTTP checker find, and why are its findings not proof of exploitability?
The checker found that the test website is served over plain HTTP with no encryption, it identifies itself in the Server header as FSCT8561LabHTTP/1.0, its session cookie is set with none of the Secure, HttpOnly or SameSite flags, and that the Content-Security-Policy and X-Content-Type-Options headers are missing. The Strict-Transport-Security header is also missing, but that is reported as a note only, because HSTS applies to HTTPS sites and this training site is HTTP only.

This is not proof of exploitability because the checker only reads what the server volunteers in its responses. It sends normal requests and inspects headers. It never attempts an attack, never submits input to be executed, and never observes the server side code or configuration

beyond the response headers. A missing header means a defence is absent, so an attack like this is not blocked by that layer. It does not show that the attack would succeed against this application, which depends on how the application is actually built and used. These are most likely configuration weaknesses to review, and not demonstrated vulnerabilities.

12. What are the confidentiality and replay limitations of a plain TCP connection, even with TOTP MFA?

The TOTP code proves that the user holds the shared secret at that moment. It does not protect the connection itself, and two limits follow.

Confidentiality

- Nothing on the connection is encrypted. The username, the password and the code all cross the network as readable text, along with every command and every result, including the PCAP analysis. Anyone that can observe the traffic on the same network can read it all. The second factor does not change this, because the code is sent in the clear like everything else.

Replay

- The password is static and the captured password can be replayed at any later time to pass the first step. The code is time based, so a captured code stops working after its time window, which the server widens slightly to allow for clock differences. Within that window, a captured code is still valid and can be reused by whoever captured it. There is also no protection against an attacker relaying messages in real time between the real client and the real server, because nothing binds the login to this specific connection.

13. What would you improve before deploying this application in production?

Encrypt the transport

- Put the connection inside TLS so credentials, codes and results are confidential and tampering and real time relaying are resisted.

Store passwords properly

- Replace the unsalted SHA-256 hash with a salted slow password hash such as Argon2, bcrypt, scrypt or PBKDF2, and give each user their own account and their own TOTP secret.

Enrol codes properly

- Stop printing the current code in the server console. In production the secret is enrolled into an authenticator app once, and no component ever displays a live code.

Slow down guessing across connections.

- Add account level lockout or delays and rate limits that survive reconnecting, instead of only counting attempts inside one connection.

Add real sessions.

- Issue a session token after login with an idle timeout, rather than trusting the connection state alone.

Protect the supporting pieces

- Store the log somewhere append only and access controlled, run the PCAP parser in a sandbox because capture files are untrusted input, and require an authorization check and approval before the scanner or checker is ever pointed at anything beyond localhost.